6.3.2 Phenomena

For reasonable transaction throughput values, it makes sense to imply transaction *serializability*. As the incoming traffic grows, the price for strict data integrity becomes too high, and this is the primary reason for having multiple isolation levels. Relaxing serializability guarantees may generate data integrity anomalies, which are also referred as *phenomena*.

The SQL-92 standard introduced three phenomena that can occur when moving away from a serializable transaction schedule:

- *dirty read*
- *non-repeatable read*
- *phantom read.*

In reality, there are other phenomena that can occur due to transaction interleaving, as the famous paper *A Critique of ANSI SQL Isolation Levels*[4] describes:

- *dirty write*
- *read skew*
- *write skew*
- *lost update.*

> Choosing a certain isolation level is a trade-off between increasing concurrency and acknowledging the possible anomalies that might occur.
>
> Scalability is undermined by contention and coherency costs. The lower the isolation level, the less locking (or multi-version transaction abortions), and the more scalable the application gets.
>
> However, a lower isolation level allows more phenomena, and the data integrity responsibility is shifted from the database side to the application logic, which must ensure that it takes all measures to prevent or mitigate any such data anomaly.

Before jumping to isolation levels, it is better to understand what's behind each particular phenomenon and how it can affect data integrity. When choosing a given transaction isolation level, understanding phenomena becomes fundamental to taking the right decision,

[4] http://research.microsoft.com/apps/pubs/default.aspx?id=69541

6.3.2.1 Dirty write

A dirty write happens when two concurrent transactions are allowed to modify the same row at the same time. As previously mentioned, all changes are applied to the actual database object structures, which means that the second transaction simply overwrites the first transaction pending change.

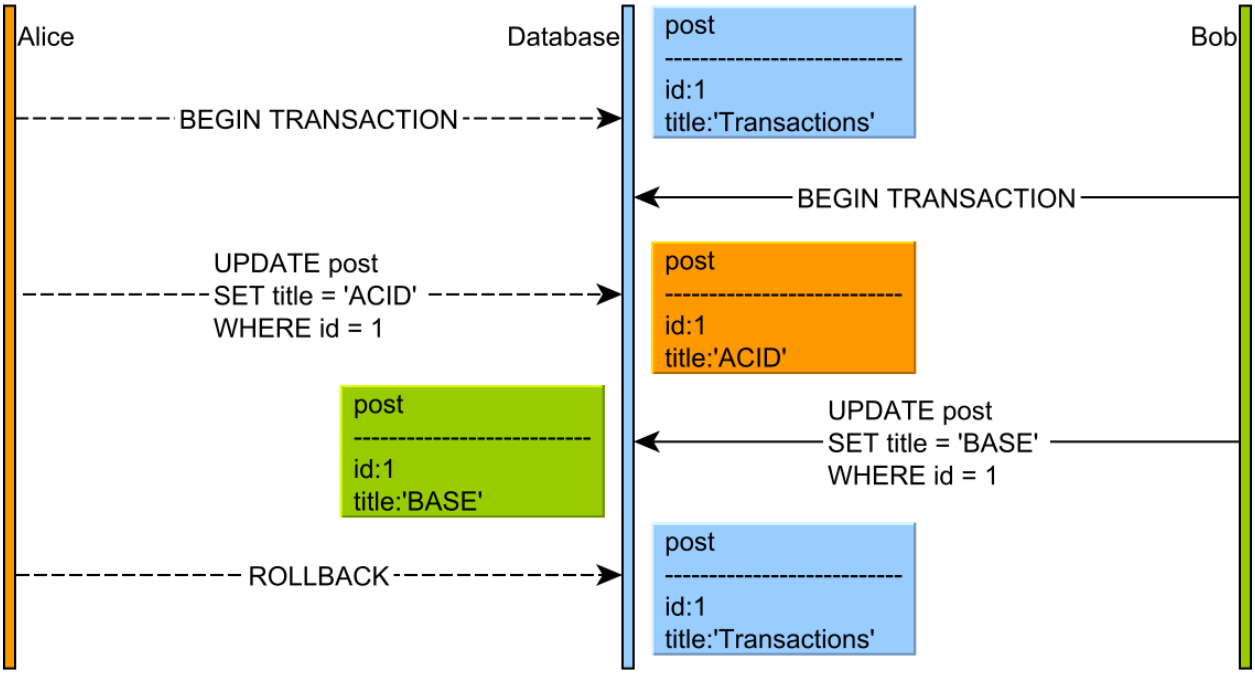

Figure 6.4: Dirty write

If the two transactions commit, one transaction will silently overwrite the other transaction, causing a *lost update*. Another problem arises when the first transaction wants to roll back. The database engine would have to choose one of the following action paths:

- It can restore the row to its previous version (as it was before the first transaction changed it), but then it overwrites the second transaction uncommitted change.
- It can acknowledge the existence of a newer version (issued by the second transaction), but then, if the second transaction has to roll back, its previous version will become the uncommitted change of the first transaction.

If the database engine did not prevent dirty writes, guaranteeing rollbacks would not be possible. Because atomicity cannot be implemented in the absence of reliable rollbacks, all database systems must, therefore, prevent dirty writes.

Although the SQL standard does not mention this phenomenon, even the lowest isolation level (*Read Uncommitted*) can prevent it.

6.3.2.2 Dirty read

As previously mentioned, all database changes are applied to the actual data structures (memory buffers, data blocks, indexes). A dirty read happens when a transaction is allowed to read the uncommitted changes of some other concurrent transaction. Taking a business decision on a value that has not been committed is risky because uncommitted changes might get rolled back.

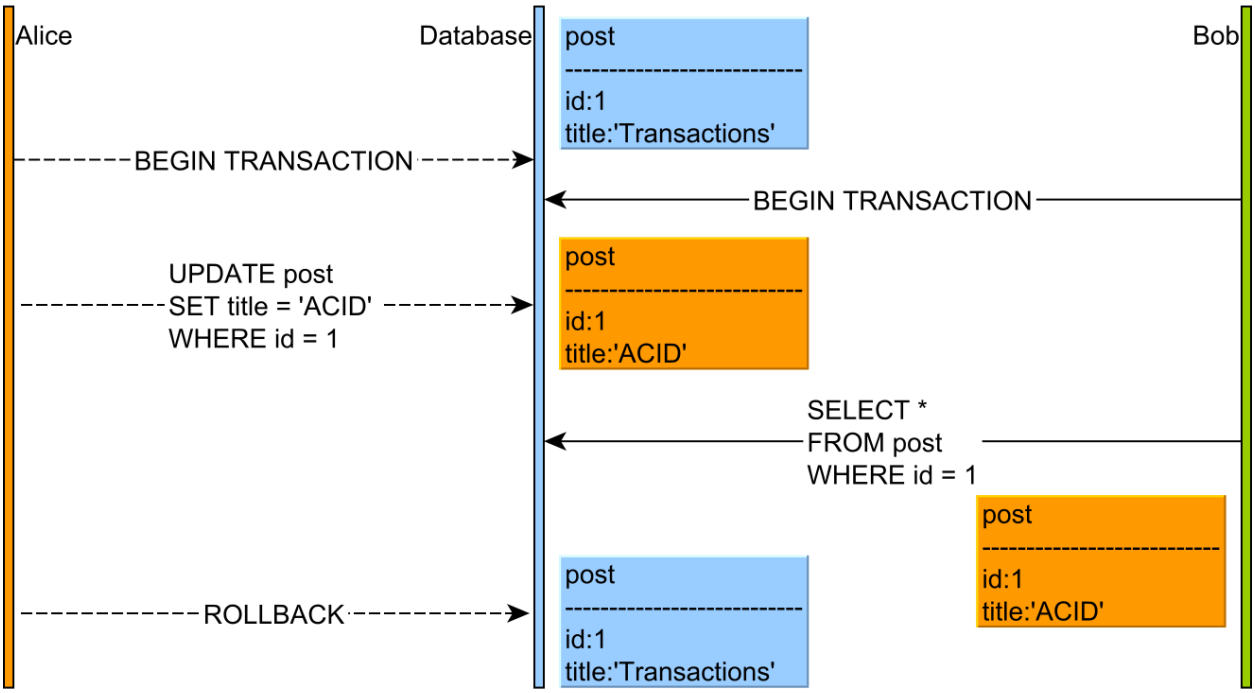

Figure 6.5: Dirty read

This anomaly is only permitted by the Read Uncommitted isolation level, and, because of the serious impact on data integrity, most database systems offer a higher default isolation level.

To prevent dirty reads, the database engine must hide uncommitted changes from all other concurrent transactions. Each transaction is allowed to see its own changes because otherwise the read-your-own-writes consistency guarantee is compromised.

If the underlying database uses 2PL (Two-Phase Locking), the uncommitted rows are protected by write locks which prevent other concurrent transactions from reading these records until they are committed.

When the underlying database uses MVCC (Multi-Version Concurrency Control), the database engine can use the undo log which already captures the previous version of every uncommitted record, to restore the previous value in other concurrent transaction queries. Most database systems optimize the before image restoring process, therefore lowering its overhead on the overall application performance.

Read Uncommitted is rarely needed (non-strict reporting queries where dirty reads are acceptable), so Read Committed is usually the lowest practical isolation level.

6.3.2.3 Non-repeatable read

If one transaction reads a database row without applying a shared lock on the newly fetched record, then a concurrent transaction might change this row before the first transaction has ended.

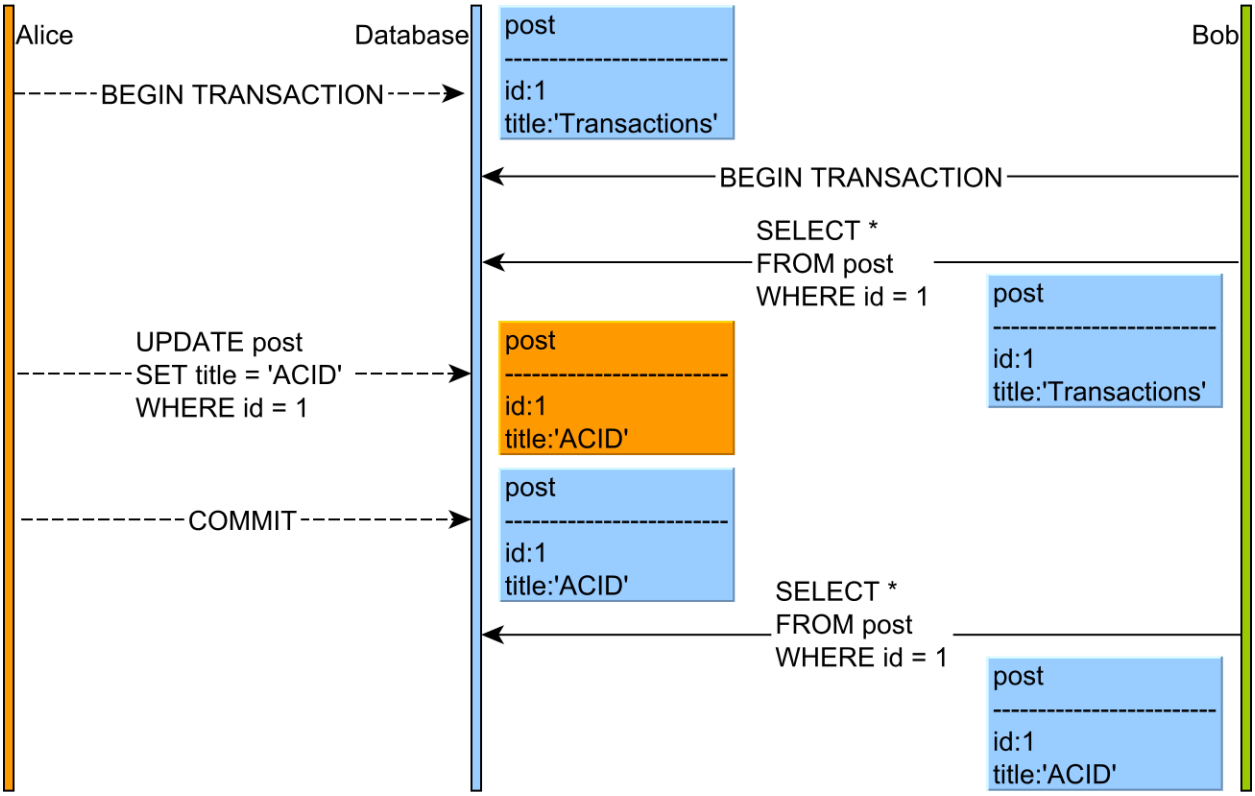

Figure 6.6: Non-repeatable read

This phenomenon is problematic when the current transaction makes a business decision based on the first value of the given database row (a client might order a product based on a stock quantity value that is no longer a positive integer).

Most database systems have moved to a Multi-Version Concurrency Control model, and shared locks are no longer mandatory for preventing non-repeatable reads. By verifying the current row version, a transaction can be aborted if a previously fetched record has changed in the meanwhile.

Repeatable Read and Serializable prevent this anomaly by default. With Read Committed, it is possible to avoid non-repeatable (fuzzy) reads if the shared locks are acquired explicitly (e.g. `SELECT FOR SHARE`).

Some ORM frameworks (e.g. JPA/Hibernate) offer application-level repeatable reads. The first snapshot of any retrieved entity is cached in the currently running *Persistence Context*. Any successive query returning the same database row is going to use the very same object that was previously cached. This way, the fuzzy reads may be prevented even in Read Committed isolation level.

6.3.2.4 Phantom read

If a transaction makes a business decision based on a set of rows satisfying a given predicate, without range locks, a concurrent transaction might insert a record matching that particular predicate.

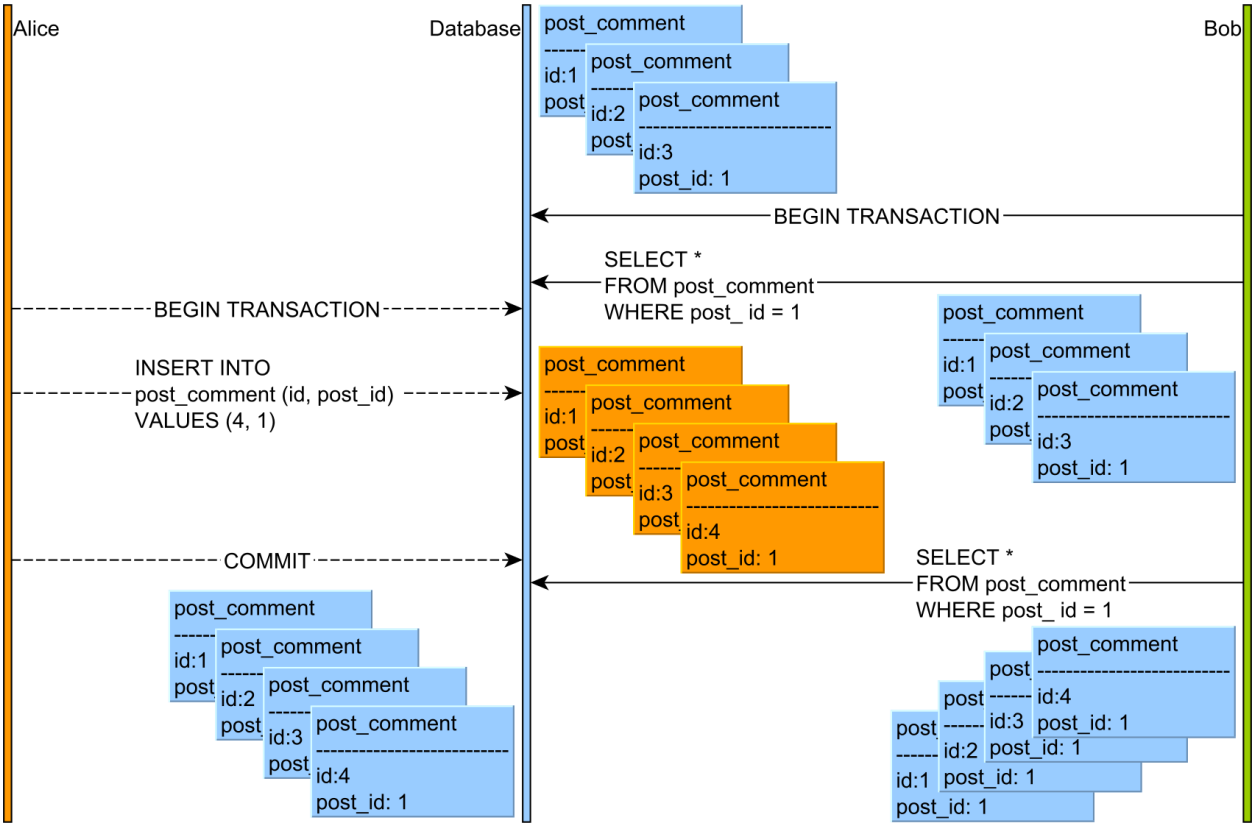

Figure 6.7: Phantom read

The SQL standard says that Phantom Read occurs if two consecutive query executions render different results because a concurrent transaction has modified the range of records in between the two calls. Although providing consistent reads is a mandatory requirement for serializability, that is not sufficient. For instance, one buyer might purchase a product without being aware of a better offer that was added right after the user has finished fetching the offer list.

The 2PL-based Serializable isolation prevents Phantom Reads through the use of predicate locking. On the other hand, MVCC database engines address the Phantom Read anomaly by returning consistent snapshots. However, a concurrent transaction can still modify the range of records that was read previously. Even if the MVCC database engine introspects the transaction schedule, the outcome is not always the same as a 2PL-based implementation. One such example is when the second transaction issues an insert without reading the same range of records as the first transaction. In this particular use case, some MVCC database engines will not end up rolling back the first transaction.

6.3.2.5 Read skew

Read skew is a lesser known anomaly that involves a constraint on more than one database tables. In the following example, the application requires the *post* and the *post_details* be updated in sync. Whenever a *post* record changes, its associated *post_details* must register the user who made the current modification.

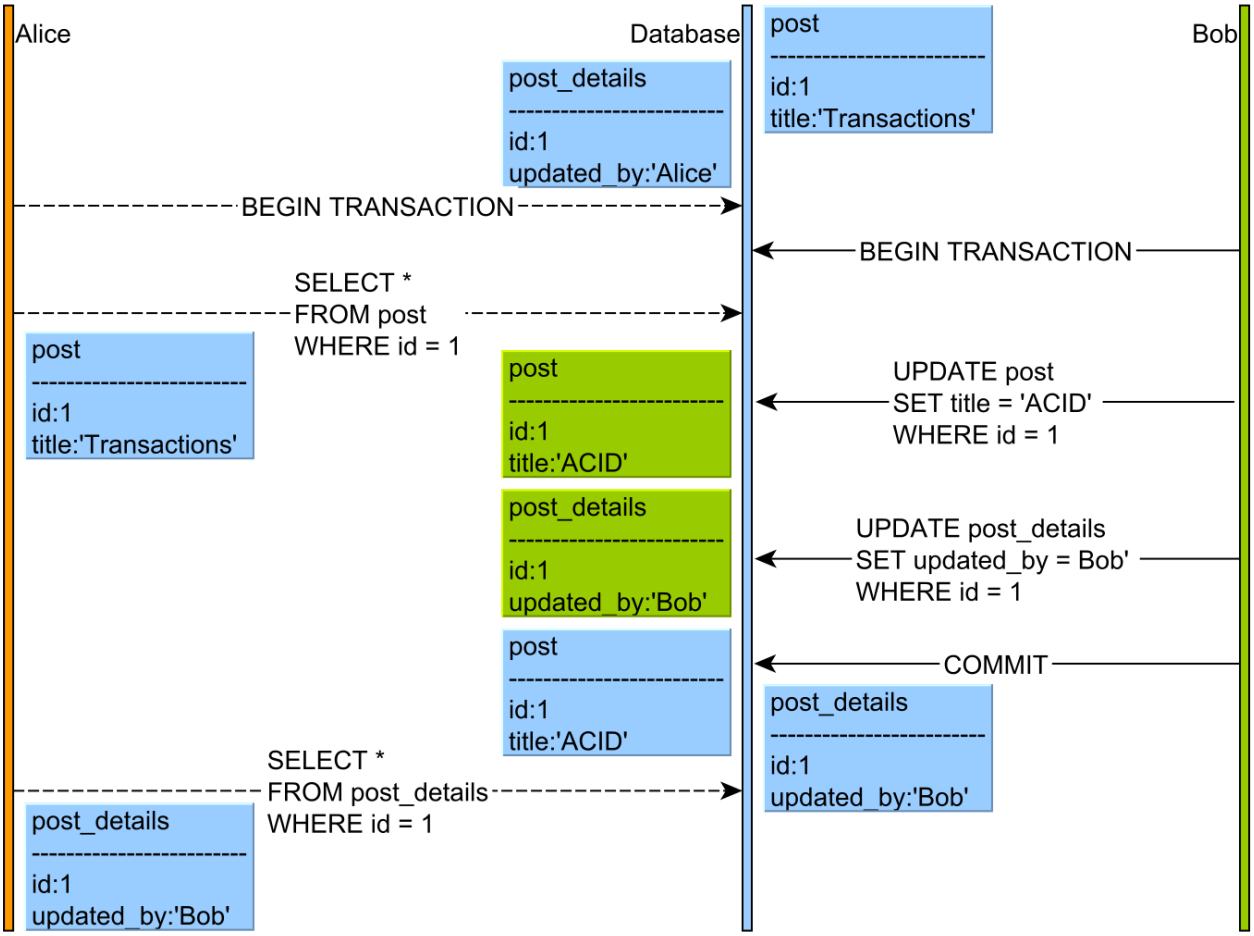

Figure 6.8: Read skew

In between selecting the *post* and the *post_details* rows, a second transaction sneaks in and manages to update both records. The first transaction sees an older version of the *post* row and the latest version of the associated *post_details*. Because of this read skew, the first transaction assumes that this particular *post* was updated by *Bob*, although, in fact, it is an older version updated by *Alice*.

Like with non-repeatable reads, there are two ways to avoid this phenomenon:

- The first transaction can acquire shared locks on every read, therefore preventing the second transaction from updating these records.
- The first transaction can be aborted upon validating the commit constraints (when using an MVCC implementation of the Repeatable Read or Serializable isolation levels).

6.3.2.6 Write skew

Like read skew, this phenomenon involves disjoint writes over two different tables that are constrained to be updated as a unit. Whenever the *post* row changes, the client must update the *post_details* with the user making the change.

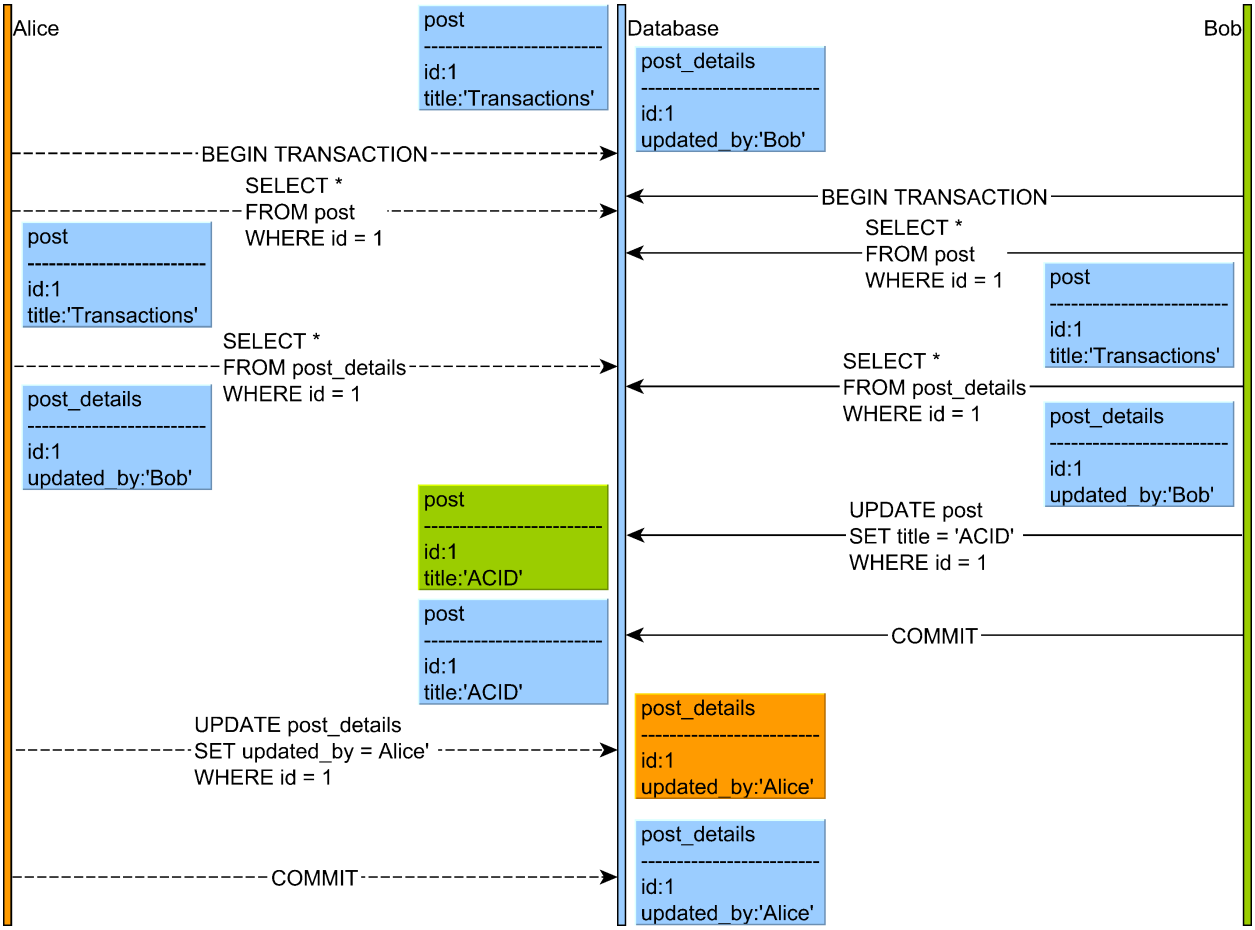

Figure 6.9: Write skew

Both *Alice* and *Bob* selects the *post* and its associated *post_details* record. If write skew is allowed, *Alice* and *Bob* can update these two records separately, therefore breaking the constraint.

Like with non-repeatable reads, there are two ways to avoid this phenomenon:

- The first transaction can acquire shared locks on both entries, therefore preventing the second transaction from updating these records.
- The database engine can detect that another transaction has changed these records, and so it can force the first transaction to roll back (under an MVCC implementation of Repeatable Read or Serializable).

6.3.2.7 Lost update

This phenomenon happens when a transaction reads a row while another transaction modifies it prior to the first transaction to finish. In the following example, *Bob*'s update is silently overwritten by *Alice*, who is not aware of the record update.

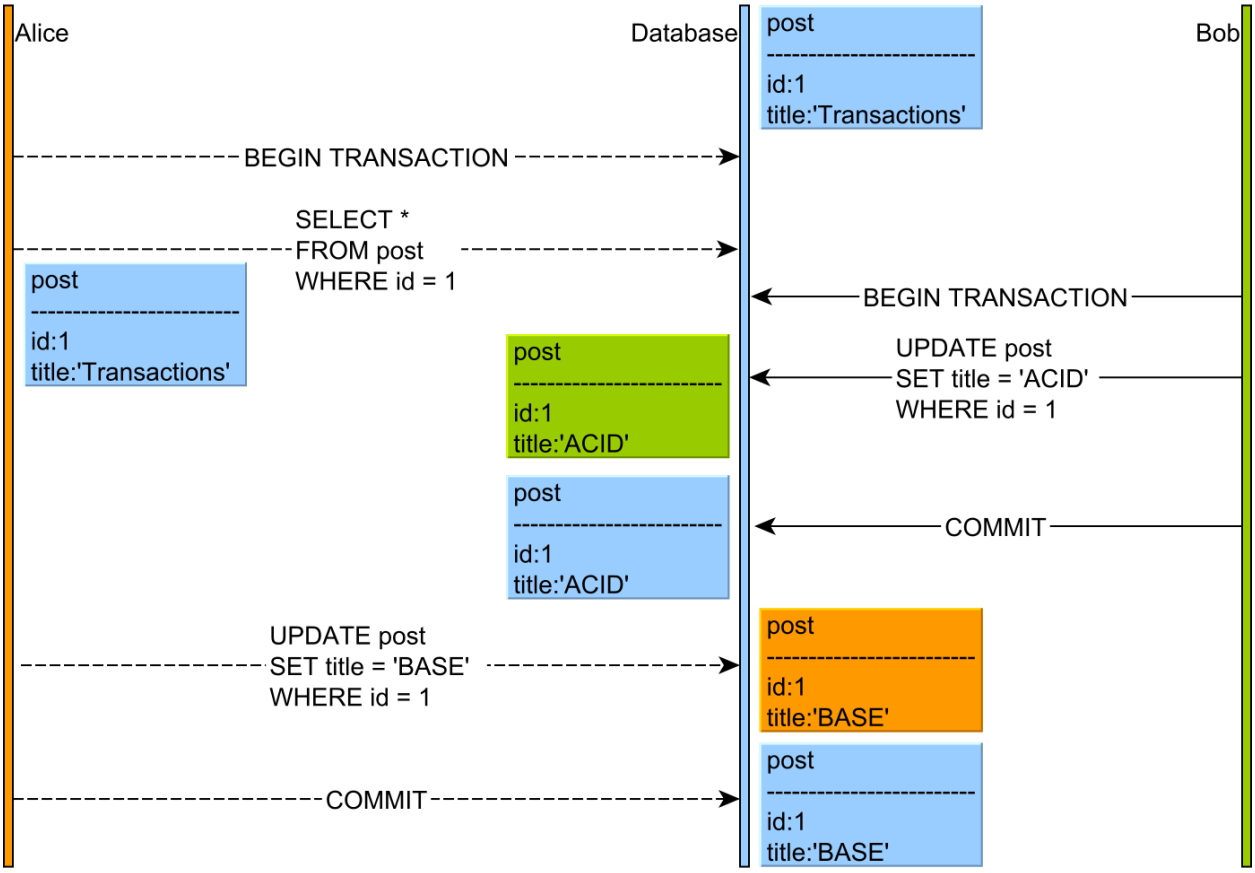

Figure 6.10: Lost update

This anomaly can have serious consequences on data integrity (a buyer might purchase a product without knowing the price has just changed), especially because it affects Read Committed, the default isolation level in many database systems.

Traditionally, Repeatable Read protected against lost updates since the shared locks could prevent a concurrent transaction from modifying an already fetched record. With MVCC, the second transaction is allowed to make the change, while the first transaction is aborted when the database engine detects the row version mismatch (during the first transaction commit).

Most ORM tools, such as Hibernate, offer application-level optimistic locking, which automatically integrates the row version whenever a record modification is issued. On a row version mismatch, the update count is going to be zero, so the application can roll back the current transaction, as the current data snapshot is stale.

6.3.3 Isolation levels

As previously stated, Serializable is the only isolation level to provide a truly ACID transaction interleaving. However, serializability comes at a price as locking introduces contention, which, in turn, limits concurrency and scalability. Even in multi-version concurrency models, serializability may require aborting too many transactions that are affected by phenomena.

For this purpose, the SQL-92 version introduced multiple isolation levels, and the database client has the option of balancing concurrency against data correctness. Each isolation level is defined in terms of the minimum number of phenomena that it must prevent, and so the SQL standard introduces the following transaction isolation levels:

Table 6.1: Standard isolation levels

Isolation Level	Dirty read	Non-repeatable read	Phantom read
Read Uncommitted	Yes	Yes	Yes
Read Committed	No	Yes	Yes
Repeatable Read	No	No	Yes
Serializable	No	No	No

Without an explicit setting, the JDBC driver uses the default isolation level, which can be introspected using the `getDefaultTransactionIsolation()`[5] method of the `DatabaseMetaData` object:

```
int level = connection.getMetaData().getDefaultTransactionIsolation();
```

The default isolation level can be changed using the `setTransactionIsolation(int level)`[6] `Connection` method.

```
connection.setTransactionIsolation(Connection.TRANSACTION_SERIALIZABLE);
```

> Even if ACID properties imply a serializable schedule, most relational database systems use a lower default isolation level instead:
>
> - Read Committed (Oracle, SQL Server, PostgreSQL)
> - Repeatable Read (MySQL).

The following sections go through each particular transaction isolation level and demonstrate the actual list of phenomena that are prevented by a given database system.

[5] http://docs.oracle.com/javase/8/docs/api/java/sql/DatabaseMetaData.html#getDefaultTransactionIsolation--
[6] http://docs.oracle.com/javase/8/docs/api/java/sql/Connection.html#setTransactionIsolation-int-

6.3.3.1 Read Uncommitted

Table 6.2: Read Uncommitted phenomena occurrence

Phenomena	SQL Server	PostgreSQL	MySQL
Dirty Write	No	No	No
Dirty Read	Yes	No	Yes
Non-Repeatable Read	Yes	Yes	Yes
Phantom Read	Yes	Yes	Yes
Read Skew	Yes	Yes	Yes
Write Skew	Yes	Yes	Yes
Lost Update	Yes	Yes	Yes

Oracle

Dirty reads are not allowed, and so the lowest isolation level is Read Committed.

The JDBC driver will even throw an exception if the client tries to set the Read Uncommitted isolation on the current `Connection`.

SQL Server

Read Uncommitted only protects against dirty writes, all other phenomena being allowed. When using Read Uncommitted, there is no exclusive lock associated with a given SQL modification, so uncommitted changes are available to other concurrent transactions even before they get committed. If the risk of dirty reads can be assumed, avoiding exclusive locks may speed up reporting queries, especially when scanning large amounts of data.

For lock-based concurrency control mechanisms, Read Uncommitted is worth considering if the risk of dirty reads is a much smaller issue than locking a large portion of a database table. Because MVCC avoids reader-writer and writer-reader locking, it might not exhibit a considerable performance enhancement from permitting dirty reads.

PostgreSQL

Like Oracle, PostgreSQL does not allow dirty reads, the lowest isolation level being Read Committed.

When choosing Read Uncommitted, the JDBC driver silently falls back to Read Committed.

MySQL

Although it uses MVCC, InnoDB implements Read Uncommitted so that dirty reads are permitted. As an optimization, each query is spared from rebuilding the previously committed versions (using the rollback segments) of the currently scanned records (in case they have been recently modified).

6.3.3.2 Read Committed

Read Committed is one of the most common isolation levels, and it behaves consistently across multiple relational database systems or various concurrency control models.

Many database systems choose Read Committed as the default isolation level because it delivers the best performance while preventing fatal anomalies such as dirty writes and dirty reads. However, performance has its price as Read Committed permits many anomalies that might lead to data corruption.

Table 6.3: Read Committed phenomena occurrence

Phenomena	Oracle	SQL Server	SQL Server MVCC	PostgreSQL	MySQL
Dirty Write	No	No	No	No	No
Dirty Read	No	No	No	No	No
Non-Repeatable Read	Yes	Yes	Yes	Yes	Yes
Phantom Read	Yes	Yes	Yes	Yes	Yes
Read Skew	Yes	Yes	Yes	Yes	Yes
Write Skew	Yes	Yes	Yes	Yes	Yes
Lost Update	Yes	Yes	Yes	Yes	Yes

Oracle

Every statement has a start timestamp, which is used to create a database snapshot relative to this particular point-in-time. This way, writers can still update the currently selected records, and the database can simply reconstruct the previous versions that were available when the query started. Subsequent query executions can return different row versions, so non-repeatable reads are permitted.

When two transactions attempt to update the same record, the first one locks the record to prevent dirty writes. The second transaction must wait until the first transaction releases the lock (either commit or rollback), and the statement filtering criteria are reevaluated against latest data.

PostgreSQL

Like Oracle, every query sees a database snapshot as of the beginning of the currently running query. Because shared locks are not used to protect previously read records from being modified, Read Committed allows a large spectrum of data anomalies.

Exclusive locks prevent write-write conflicts, so when two transactions update the same record, the second one waits for the first transaction to release its locks. When the second transaction resumes its execution, if the filtering criteria are still relevant, it might overwrite the first transaction modifications, therefore causing lost updates.

MySQL

Query-time snapshots are used to isolate statements from other concurrent transactions. When explicitly acquiring shared or exclusive locks or when issuing update or delete statements (which acquire exclusive locks to prevent dirty writes), if the selected rows are filtered by unique search criteria (e.g. primary key), the locks can be applied to the associated index entries.

Prior to 5.7[a], if the modifying statements used a range filter and the search criteria took advantage of a unique index scan, then the database could use a gap or a next-key lock (therefore protecting against phantom reads as well). Statement-based replication is not available for Read Committed, so the application must use the row-based binary logging instead.

[a] https://dev.mysql.com/doc/refman/5.7/en/set-transaction.html#idm140311316367072

SQL Server

By default, SQL statements use shared locks to prevent other transactions from modifying the currently fetched records. The locks are released by the time the query finishes executing. When activating Read Committed Snapshot Isolation, the database does not use shared locks anymore, and each query selects the row version as it was when the query started.

6.3.3.3 Repeatable Read

One of the least compliant isolation levels, Repeatable Read implementation details leak into its phenomena prevention spectrum:

Table 6.4: Repeatable Read phenomena occurrence

Phenomena	SQL Server	PostgreSQL	MySQL
Dirty Write	No	No	No
Dirty Read	No	No	No
Non-Repeatable Read	No	No	No
Phantom Read	Yes	No	No
Read Skew	No	No	No
Write Skew	No	Yes	Yes
Lost Update	No	No	Yes

Oracle

The Repeatable Read isolation is not supported at all, and the JDBC driver will throw an exception if the client tries to set it explicitly.

SQL Server

For every row the client reads, the current transaction acquires a shared lock that prevents any other transaction from concurrently modifying it. The shared locks are released when the transaction either commits or rolls back.

PostgreSQL

The Repeatable Read is implemented using Snapshot Isolation[a], so not only fuzzy reads are prevented, but even phantom reads are prohibited as well. Instead of using locking, the PostgreSQL MVCC implementation allows conflicts to occur, but it aborts any transaction whose guarantees do not hold anymore.

[a] https://en.wikipedia.org/wiki/Snapshot_isolation

MySQL

Every transaction can only see rows as if they were when the current transaction started. This prevents non-repeatable reads, but it still allows lost updates and write skews.

6.3.3.4 Serializable

Serializable is supposed to provide a transaction schedule, whose outcome, even in spite of statement interleaving, is equivalent to a serial execution.

Even if the concurrency control mechanism is lock-based or it manages multiple record versions, it must prevent all phenomena to ensure serializable transactions. Preventing all phenomena mentioned by the SQL standard (dirty reads, non-repeatable reads and phantom reads) is not enough, and Serializable must protect against lost update, read skew and write skew as well.

In practice, the concurrency control implementation details leak, and not all relational database systems provide a truly Serializable isolation level (some data integrity anomalies might still occur).

Table 6.5: Serializable phenomena occurrence

Phenomena	Oracle	SQL Server	SQL Server MVCC	PostgreSQL	MySQL
Dirty Write	No	No	No	No	No
Dirty Read	No	No	No	No	No
Non-Repeatable Read	No	No	No	No	No
Phantom Read	No	No	No	No	No
Read Skew	No	No	No	No	No
Write Skew	Yes	No	Yes	No	No
Lost Update	No	No	No	No	No

Oracle

The Serializable isolation level is, in fact, an MVCC implementation of the Snapshot Isolation concurrency control mechanism. Like the Repeatable Read isolation in PostgreSQL, Oracle cannot prevent write skews, meaning it cannot provide a truly serializable transaction.

SQL Server

The Serializable isolation level is based on 2PL, and all phenomena are therefore prevented. The MVCC-based Snapshot Isolation is close to Oracle Serializable and PostgreSQL Repeatable Read, and so it allows write skews.

PostgreSQL

To overcome the Snapshot Isolation limitations, PostgreSQL has developed the Serializable Snapshot Isolation (SSI), which provides true serializable transactions. Because SSI is still an MVCC implementation, PostgreSQL monitors the transaction schedule and detects *possible* serializability anomalies.

The current implementation may detect *false positives*[a], and some transactions might get aborted even if they did not really break transaction serializability. Only the Precisely Serializable Snapshot Isolation (PSSI) model can eliminate all false positives, but the performance penalty being too high, the database implementers stuck to SSI instead.

[a] http://drkp.net/papers/ssi-vldb12.pdf

MySQL

The Serializable isolation builds on top of Repeatable Read with the difference that every record that gets selected is protected with a shared lock as well. The lock-based approach allows MySQL to prevent the write skew phenomena, which is prevalent among many Snapshot Isolation implementations.

6.4 Durability

When purchasing an airline ticket, the money is withdrawn from the bank account, and a seat is reserved for the given buyer. Assuming that, right after the ticket is purchased, the airline reservation system crashes, all the previously processed transactions must hold true even after the system restarts. If the system does not enforce this requirement, the registered ticket might vanish, and the buyer is possibly left with a debited account and no ticket at all.

> Durability ensures that all committed transaction changes become permanent.

Durability allows system recoverability, and, to some extent, it is similar to the rolling back mechanism.

> **What about undo logs?**
>
> To support transaction rollbacks and to rebuild previous versions in MVCC systems, the database system already records the current modifications (including uncommitted changes) in the undo log. However, recoverability needs committed changes only, and, because the obsolete undo segments might be frequently recycled, the undo log alone is not suitable for recoverability.

When a transaction is committed, the database persists all current changes in an append-only, sequential data structure commonly known as the *redo log*.

> **Oracle**
>
> The *redo log*[a] consists of multiple redo records, each one containing *change vectors*, which capture the actual data block changes. For performance reasons, the redo records are stored in a buffer, and the Log Writer flushes the in-memory records to the current active redo log file. At any given time, Oracle has, at least, two redo files, but only one of them is active and available for collecting the log buffer entries. When a transaction is committed, the database flushes the buffer and changes become persisted.
>
> If the buffer fills, Oracle will flush it along with any uncommitted changes, which can be removed if their associated transaction is rolled back.
>
> [a] https://docs.oracle.com/database/121/ADMIN/onlineredo.htm#ADMIN11302

SQL Server

Unlike Oracle, SQL Server combines both the undo and the redo log into a single data structure called *transaction log*. By default, when a transaction is committed, all the associated transaction log entries are flushed to the disk before returning the control back to the client.

SQL Server 2014 added support for configurable durability[a]. The log entry flushing can be delayed, which can provide better I/O utilization and lower transaction response times. If the system crashes, all the unflushed log entries will be wiped out from memory. Asynchronous flushing is, therefore, appropriate only when data loss is tolerated.

[a] https://msdn.microsoft.com/en-us/library/dn449490.aspx

PostgreSQL

Statement changes are captured in the *Write-Ahead Log* (WAL)[a]. The log entries are buffered in memory and flushed on every transaction commit.

Because their state can be restored from the WAL during recovery, the cached data pages and index entries need not be flushed for every transaction (therefore optimizing I/O utilization). Ever since 9.1[b], PostgreSQL supports configurable durability, so the WAL can also be flushed asynchronously.

[a] http://www.postgresql.org/docs/current/static/wal-intro.html
[b] http://www.postgresql.org/docs/current/static/non-durability.html

MySQL

All the *redo log* entries associated with a single transaction are stored in the *mini transaction buffer* and flushed at once into the *global redo buffer*. The global buffer is flushed to disk during commit. By default, there are two log files which are used alternatively.

Flushing is done synchronously by default, but it can be switched to an asynchronous mode via the innodb_flush_log_at_trx_commit[a] parameter.

[a] http://dev.mysql.com/doc/refman/5.7/en/innodb-parameters.html#sysvar_innodb_flush_log_at_trx_commit

Since durability is very important for business operations, it is better to stick to the synchronous flushing mechanism.

Delaying durability guarantees becomes a valid option only when data loss is tolerated by business requirements and the redo log flushing is a real performance bottleneck.

6.5 Read-only transactions

The JDBC `Connection` defines the `setReadOnly(boolean readOnly)`[7] method which can be used to *hint* the driver to apply some database optimizations for the upcoming read-only transactions. This method should not be called in the middle of a transaction because the database system cannot turn a read-write transaction into a read-only one (a transaction must start as read-only from the very beginning).

Oracle

According to the JDBC driver documentation[a], the database server does not support read-only transaction optimizations. Even when the read-only `Connection` status is set to `true`, modifying statements are still permitted, and the only way to restrict such statements is to execute the following SQL command:

```
connection.setAutoCommit(false);
try(CallableStatement statement = connection.prepareCall(
   "BEGIN SET TRANSACTION READ ONLY; END;")) {
    statement.execute();
}
```

The `SET TRANSACTION READ ONLY` command must run after disabling the auto-commit status, as otherwise it is only applied for this particular statement only.

[a] https://docs.oracle.com/database/121/JJDBC/apxtips.htm#JJDBC28956

[7] http://docs.oracle.com/javase/8/docs/api/java/sql/Connection.html#setReadOnly%28boolean%29

SQL Server

Like Oracle, the read-only `Connection` does not propagate to the database engine, and the only way to disable SQL modifications is to use a separate account, restricted to viewing data only.

Setting the `ApplicationIntent=ReadOnly`[a] connection property does not prevent the JDBC driver from executing modifying statements on a read-only `Connection`. This property has the purpose of routing read-write and read-only connections to replica nodes instead.

[a] https://msdn.microsoft.com/en-us/library/gg471494.aspx

PostgreSQL

An exception is thrown when executing a modifying statement on a `Connection` whose read-only status was set to `true`.

The database engine optimizes read-only transactions, so the false-positives anomaly rate is reduced for the Serializable isolation level, and it allows *deferrable* serializable snapshots[a]. A deferrable snapshot is activated when executing `SET TRANSACTION SERIALIZABLE READ ONLY DEFERRABLE`. The current transaction must wait for a safe snapshot to become available, which can be executed without the risk of being aborted by a non-serializable anomaly. If the default read-write Serializable transactions are problematic when accessing large volumes of data, the deferrable snapshots might be a better alternative for long-running transactions.

[a] http://arxiv.org/pdf/1208.4179.pdf

MySQL

If a modifying statement is executed when the `Connection` is set to read-only, the JDBC driver will throw an exception.

InnoDB can optimize read-only transactions[a] because it can skip the transaction ID generation as it is not required for read-only transactions.

[a] https://dev.mysql.com/doc/refman/5.7/en/innodb-performance-ro-txn.html

6.5.1 Read-only transaction routing

Setting up a database replication environment is useful for both high-availability (a Slave can replace a crashing Master) and traffic splitting. In a Master-Slave replication topology, the Master node accepts both read-write and read-only transactions, while Slave nodes only take read-only traffic.

Oracle

The Oracle ADG (Active Data Guard) allows an enterprise application to distribute read-write traffic to the Primary node and read-only transactions to a Standby database. WebLogic Server GridLink Data Source[a] provides failover and load balancing capabilities over Oracle ADG.

[a] http://www.oracle.com/technetwork/middleware/weblogic/learnmore/1534212

SQL Server

The database Availability Group must be configured to use read-only routing, in which case the redirection is based on the `ApplicationIntent` connection property. This means that the application requires separate `DataSource(s)` for read-write and read-only connections, and transaction routing must initiate in the application service layer.

PostgreSQL

The JDBC driver defines two connection properties[a] for load balancing purposes: `loadBalanceHosts` (which is disabled by default) and `targetServerType` (`master` or `preferSlave`). To enable transaction routing, the application must do the routing itself using separate `DataSource(s)`.

[a] https://jdbc.postgresql.org/documentation/head/connect.html

MySQL

The `com.mysql.jdbc.ReplicationDriver`[a] supports transaction routing on a Master-Slave topology, the decision being made on the `Connection` read-only status basis.

[a] https://dev.mysql.com/doc/connector-j/en/connector-j-master-slave-replication-connection.html